Training a small GPT on TinyStories under a one-hour budget: a compute-optimal sweet spot

November 8, 2026

Group Composition:

1. Lara Muy Steffany Mishell (Steffany Lara), A0350565H, [NUS email]
2. Kerem Ozemre, A0[fill], [NUS email]
3. Kowalke Jeri Tonio Janek (Tonio Kowalke), A0[fill], [NUS email]
4. Haj Hassine Ahmed Ben Abdellaziz (Ahmed Haj Hassine), A0[fill], [NUS email]

Github Repository: github.com/KeremOzemre/Optimization-for-Large-Scale-Data

Abstract

We trained 20 nanoGPT language models on TinyStories from random weights, each for one hour on the same GPU: five model sizes (16 to 253M parameters) crossed with four training-data fractions (25/50/75/100% of the training split). The tokenizer, optimizer, schedule, context length, and effective batch were held fixed, so the only things that changed were model size and how much data each run could draw from. Validation loss drops steeply from `small` to `xl` (≈124M parameters) and then gets slightly worse again at `xxl` (≈253M). The reason is simple once you look at throughput: `xxl` is so slow that it only gets through about half as many tokens in the hour, so it stays undertrained. The data fraction barely moves the loss, because no run gets through even 25% of the split in the time it has. Picking the best model by validation loss gives `xl_data50` (val loss 1.5025, perplexity 4.49); on the untouched test split it reaches loss 1.4846, perplexity 4.41, and 0.524 bits per byte.

1 Compute budget declaration

Every model starts from random weights (nanoGPT initialisation, with no pretrained weights, embeddings, or tokenizer fine-tuning) and trains for exactly one hour (3600 s) of in-loop wall-clock per model. Data preparation, the final validation pass, and checkpointing all happen outside that timed loop. All 20 runs use the same hardware: a single NVIDIA H100 NVL MIG 3g.47gb slice in `bfloat16`. The same hour of wall-clock does not buy the same number of tokens for every size, and that gap turns out to drive our main result (§5).

2 Research question

Which model size and training-data fraction give the best language-modeling performance under a fixed one-hour budget on identical hardware? We hold the tokenizer, optimizer, schedule, context length, and effective batch size fixed, and vary only (a) model size and (b) the fraction of the training split used.

3 Experimental setup

- **Data:** `roneneldan/TinyStories`. We train on the official train split, and split the official validation set 80/20 into validation and test at the story level (seed 42). The test split is only touched once, for the final comparison (§5.1).
- **Tokenizer:** GPT-2 (`tiktoken`), vocabulary 50,257, with an end-of-text token after every story. Since the tokenizer never changes, per-token cross-entropy and perplexity are directly comparable across all our runs.
- **Model:** nanoGPT [3] GPT with tied input/output embeddings, causal self-attention, context length 256, and dropout 0.1.
- **Evaluation objective:** held-out cross-entropy on the test split, also reported as perplexity (exp of the loss) and bits per byte. Cross-entropy is the quantity the model is trained on, measured on unseen text, so it reads language-modeling quality directly; perplexity is its interpretable version; and bits per byte lets our numbers be compared to work that uses a different tokenizer [1]. Splitting at the story level with seed 42 keeps any single story from leaking between validation and test.

Table 1: The five model sizes. Width divides evenly by the head count; context length (256) and dropout (0.1) are shared. We adjust microbatch and accumulation to keep an effective batch of 16 sequences (4096 tokens) per optimizer step at every size.

Size	Layers	Width	Heads	Params (M)	Microbatch × accum.
`small`	4	256	4	16.08	16×1
`medium`	6	384	6	30.02	16×1
`large`	8	512	8	51.04	16×1
`xl`	12	768	12	123.75	8×2
`xxl`	16	1024	16	253.09	4×4

Data fractions: 25/50/75/100% of the training split, giving a 5×4 grid of 20 runs. **Optimizer and schedule:** AdamW (lr 3×10^{-4}, weight decay 0.1, betas 0.9/0.95, gradient clip 1.0), with a time-based 2% warmup and then cosine decay to 10% of the peak learning rate.

4 Methodology and reproducibility

The exact commands are in the repository README; the pipeline is five steps.

1. `run.py prepare` builds (or reuses) the token cache for a given data fraction.
2. `create_experiments.py` writes the per-run configs under `configs/experiments/` from the baseline YAML.
3. `run.py train` trains one config; its outputs (`run_config.json`, `metrics.csv`, `checkpoint.pt`) land in `results/`. Checkpoints and logs are mirrored to the Hugging Face Hub (`dsa4212-tinystories`).
4. We pick the model on *validation* loss, and only then score the chosen one on *test*.
5. `eval_hf.py` downloads every checkpoint and scores the full test split; `analyze.py` then aggregates all runs into `analysis/` (the tables and figures below).

5 Results

Table 2 lists all 20 runs sorted by final validation loss, the metric we select on. We read three things off these runs, shown in Figures 1–3.

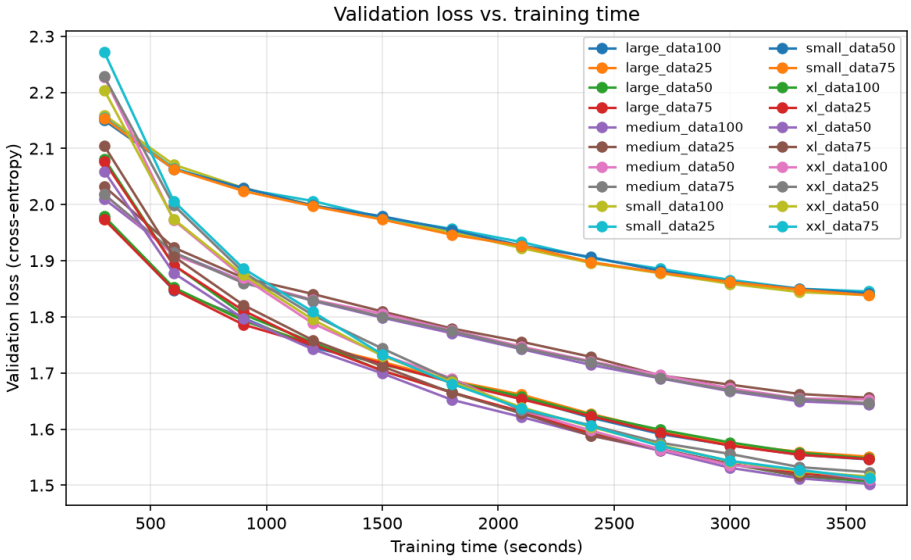


Figure 1: Validation loss over training time. The smaller models drop fastest early on, since they fit many more steps per second, and then flatten. `xl` and `xxl` start higher but pass the smaller models and are still coming down at the one-hour mark, which says they are limited by compute rather than converged.

Table 2: All 20 runs, sorted by final validation loss (the selection metric). "Tokens" is how many training tokens the run actually processed inside the one-hour budget. Test numbers are shown for completeness, but we did not use them to select the model (§5.1).

Run	Params (M)	Data%	Tokens (M)	Val loss	Val ppl	Test ppl
`xl_data50`	123.75	50	273.9	**1.5025**	**4.49**	4.41
`xl_data100`	123.75	100	279.8	1.5068	4.51	4.40
`xl_data75`	123.75	75	269.1	1.5099	4.53	4.43
`xxl_data100`	253.09	100	143.7	1.5106	4.53	4.56
`xl_data25`	123.75	25	283.3	1.5118	4.54	4.45
`xxl_data75`	253.09	75	141.1	1.5125	4.54	4.59
`xxl_data50`	253.09	50	148.2	1.5155	4.55	4.56
`xxl_data25`	253.09	25	143.2	1.5233	4.59	4.61
`large_data100`	51.04	100	562.4	1.5460	4.69	4.66
`large_data75`	51.04	75	562.0	1.5466	4.70	4.67
`large_data50`	51.04	50	557.9	1.5479	4.70	4.69
`large_data25`	51.04	25	563.2	1.5509	4.72	4.70
`medium_data100`	30.02	100	734.6	1.6444	5.18	5.16
`medium_data75`	30.02	75	738.3	1.6462	5.19	5.17
`medium_data50`	30.02	50	737.3	1.6527	5.22	5.20
`medium_data25`	30.02	25	719.1	1.6558	5.24	5.21
`small_data75`	16.08	75	940.8	1.8383	6.29	6.28
`small_data100`	16.08	100	984.3	1.8386	6.29	6.26
`small_data50`	16.08	50	985.5	1.8421	6.31	6.29
`small_data25`	16.08	25	972.0	1.8452	6.33	6.31

(1) A compute-optimal sweet spot. Bigger helps, with sharply diminishing returns, and then stops helping. Validation loss falls 1.84 → 1.64 → 1.55 → 1.50 from `small` to `xl`, but the best `xxl` run (1.5106) comes out above the best `xl` run (1.5025). `xxl` has about twice the parameters of `xl` and runs at roughly half the throughput, so it only sees about 144M tokens in the hour against 274M for `xl`, and never finishes training (Fig. 3). This is the compute-optimal picture of Hoffmann et al. [2]: for a fixed compute budget there is a best model size, and going past it spends the budget on parameters that never see enough tokens.

(2) The data fraction barely matters. Within each size the four fractions differ by about 0.01 nats, and not monotonically (Fig. 2, right). The reason is in Table 2: no run processes more than about 985M tokens, and even the 25% subset of TinyStories holds far more tokens than that. Training samples windows with replacement, so it never gets all the way through even the smallest subset. These runs are limited by compute, not by how much data they are allowed to touch.

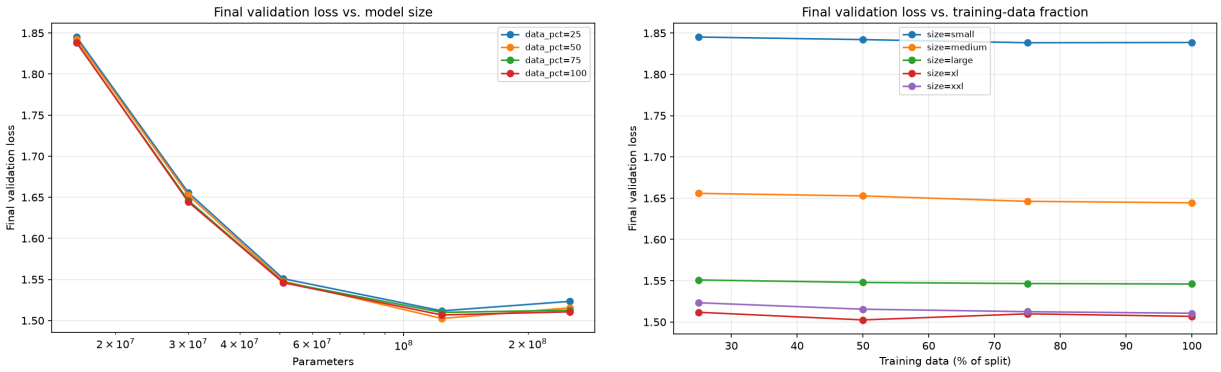


Figure 2: Left: final validation loss against model size, one line per data fraction. Loss falls steeply from `small` through `xl`, then ticks back up at `xxl`, leaving a sweet spot near 124M parameters. Right: loss against data fraction, one line per size. The lines are almost flat; within each size the four fractions differ by about 0.01 nats, and not even in a consistent direction.

(3) Test agrees with validation. The test ordering follows the validation ordering (Table 2). The full-split test losses come out a touch lower than validation, because test is scored over all 957,184 held-out tokens while in-loop validation only sampled a fixed set of windows.

5.1 Final test result

Selecting on validation loss alone gives `xl_data50` (val loss 1.5025, perplexity 4.49). Scored once on the untouched test split, it gets **test loss 1.4846, perplexity 4.41, and 0.5237 bits per byte**. The whole `xl` family is really within noise of each other (val 1.5025–1.5118; the 0.004 by which `data50` edges `data100` is run-to-run jitter), so the honest reading is "`xl` at around 124M parameters, at any data fraction" rather than 50% specifically.

5.2 Qualitative evaluation

The generated stories (temperature 0.8, top-k 40; full samples in the appendix) line up with the metrics but flatten the gaps. Even `small` writes grammatical, on-topic TinyStories. What size buys is coherence over a longer span: the `xl` winner keeps one plot and a stable cast across a whole paragraph, while `small` tends to loop ("painted a dog, and a dog") or lose the thread. The samples back up the held-out numbers; they do not stand in for them.

5.3 Estimating the compute-optimal size

Using only the runs we already have, we estimate each run's training compute with the usual approximation $C \approx 6ND$ (N parameters, D tokens processed). Fitting a parabola to the best validation loss per size in $\log_{10}$(parameters) (Fig. 4, left) puts the minimum, the

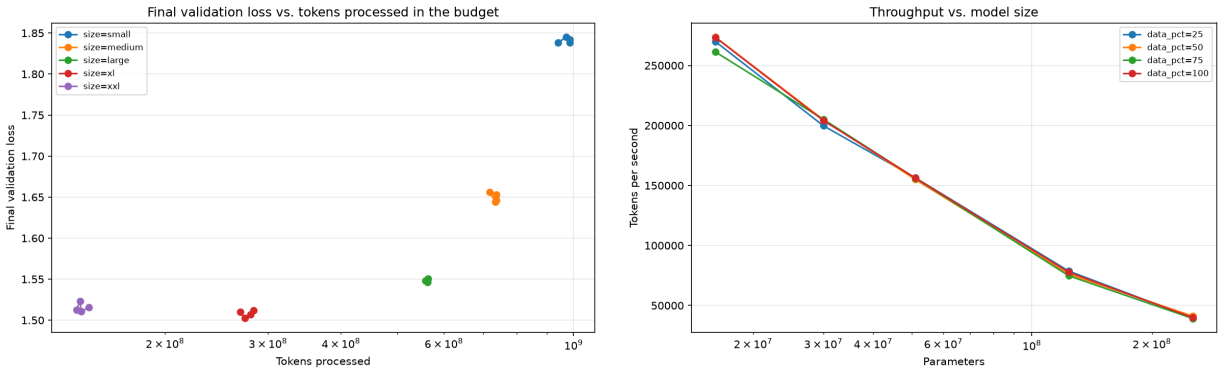


Figure 3: Throughput against tokens seen in the hour. Right: throughput collapses as the model grows, from about 270k tokens/s for `small` down to about 40k for `xxl`. Left: so the larger models get through far fewer tokens in the same hour (about 985M for `small` versus 144M for `xxl`, roughly 7 times fewer), yet still reach lower loss per token, right up until `xxl`, where too few tokens finally cost more than the extra capacity is worth.

compute-optimal size for a one-hour budget on this hardware, at $N^\star \approx 130$M parameters. That lands right on our empirical winner `xl` (124M). The loss-against-compute view (Fig. 4, right) shows a second effect: even though every run got the same hour of wall-clock, the compute it actually delivered grows with size (about 9×10^{16} FLOPs for `small` up to about 2.2×10^{17} for `xxl`), because the smallest models leave the GPU half-idle. So `small` loses twice over: less capacity, and less compute squeezed out of the same hour.

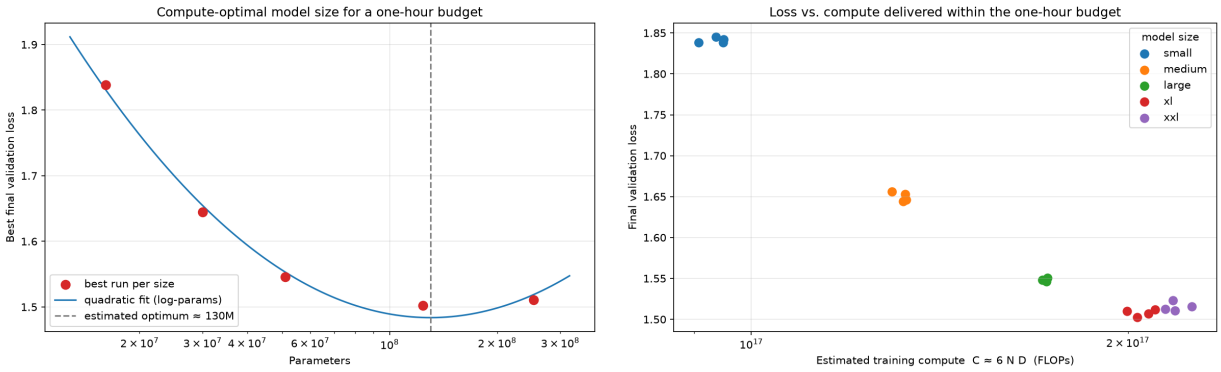


Figure 4: Left: best validation loss per size against parameters, with a quadratic fit in log-parameters whose vertex estimates the compute-optimal size ($N^\star \approx 130$M) for the one-hour budget, within noise of the empirical winner `xl`. Right: loss against estimated compute $C \approx 6ND$; the smallest models deliver the least compute despite the identical wall-clock budget. The fit rests on five points, so read it as indicative, not precise.

5.4 Varying the budget and the context length

The main grid holds two things fixed that are worth poking at: the one-hour budget and the 256-token context. We changed each on its own.

Budget. If the best size really comes out of the budget, then a smaller budget should want a smaller model. We reran `large`, `xl`, and `xxl` at 30 minutes instead of an hour. The winner moves. At 30 minutes `large` (51M) takes the lead with validation loss 1.580, ahead of `xl` (1.633) and `xxl` (1.680), whereas at a full hour `xl` was best (Fig. 5). Halving the clock pulls the best size down from 124M to 51M, which is what the compute-optimal view predicts: fewer tokens to spend, so a smaller and faster model spends them better. We compare on validation loss here, since it is measured the same way for every run.

Context length. Keeping `xl` and the one-hour budget, we raised the context from 256 to 512 tokens. Validation loss falls from 1.5068 to 1.4555 (test perplexity from 4.40 to 4.31), even though the longer-context run gets through fewer tokens in the hour (190M against 280M). On TinyStories the extra context pays for itself.

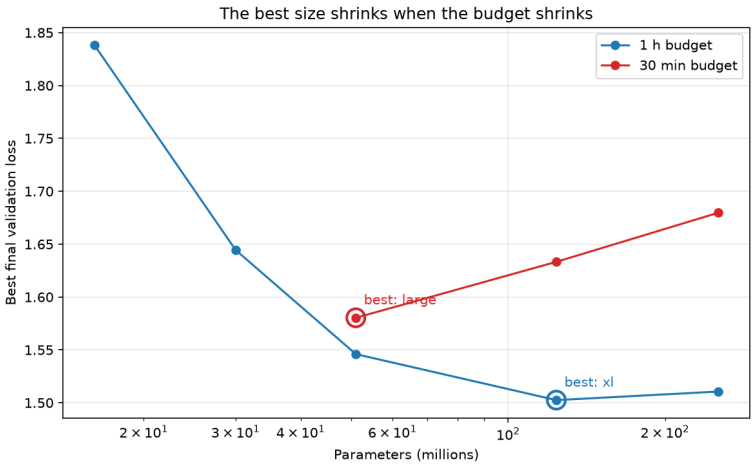


Figure 5: Best validation loss per size at two budgets. At one hour the minimum sits at `xl` (124M); at 30 minutes it moves to `large` (51M). A tighter budget prefers a smaller model.

6 Discussion

The winner wins because it sits where capacity and tokens balance for this budget: big enough to hold what the smaller models cannot, small enough to still see enough tokens in an hour to use that capacity. The whole experiment is compute-bound. Throughput, not how much data is on hand, is the binding constraint, which is why the data fraction hardly moves the loss and why `xxl` lands below `xl`. Give it a bigger budget and we would expect the

left panel of Fig. 2 to keep bending to the right: `xxl` and larger should eventually pass `xl` once they get enough tokens, and the data fraction should start to bite once a run can actually finish a subset. The 30-minute runs show the mirror image: cut the budget and the best size has already dropped to `large` (Fig. 5), and a longer context helps `xl` even though it costs tokens. A few caveats. Training samples windows with replacement, so an hour guarantees neither a full epoch nor unique coverage. The single shared recipe (learning rate, warmup, batch) is not tuned per size and may help some sizes more than others. Validation used sampled windows rather than the full test-time pass. And the TinyStories source revision is not pinned.

7 Conclusion

Under a fixed one-hour, from-scratch budget on one H100 MIG slice, the best language model on TinyStories is `xl` at around 124M parameters; chosen on validation, `xl_data50` reaches test perplexity 4.41 (0.524 bits/byte). The trade-off that matters most is throughput against capacity: because a bigger model gets through far fewer tokens in the same wall-clock time, going bigger stops paying off, and then backfires, well before the largest model we trained. How much training data a run is allowed to touch makes almost no difference once you are this compute-bound.

References

[1] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[3] Andrej Karpathy. nanogpt. https://github.com/karpathy/nanoGPT, 2023. commit 3adf61e.

A Generated story samples

The full set of samples for all 20 runs (five prompts each) lives in `analysis/generations_all.md`. As one example, both models continue "*Once upon a time*" fluently, but the `xl` winner holds a longer, more consistent story:

`small_data25`. *"Once upon a time, there was a little girl named Lily. Lily loved to play with her toys... She painted a picture of a flower and a house... Later that day, Lily's mom asked her to help. Lily was happy to help and started to decorate the room with pretty flowers."* (The thread loops back on itself.)

`xl_data50` **(selected).** *"Once upon a time, there was a little girl named Lily. Lily loved to play with her toys, especially her teddy bear. One day, Lily's mommy asked her to help decorate the house for a party... 'Mommy, can I carry my teddy bear?' she asked... They arrived at the living room and Lily's mommy gave her a big box of presents."* (One plot, moving forward.)